

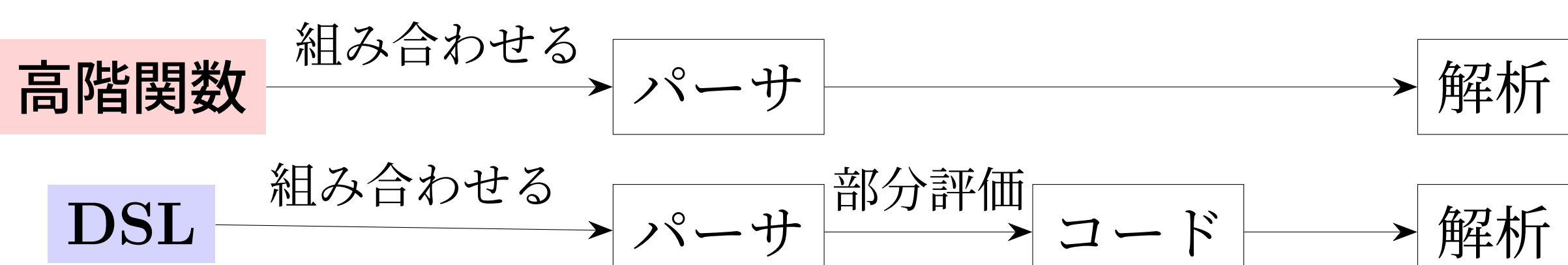
Towards Context-Sensitive Parser Combinator with Regular Language Optimization

李 張帆 亀山 幸義

筑波大学

背景・研究目標

- パーサコンビネータとは、**高階関数**を組み合わせて複雑なパーサを構築する手法である。
- しかし、**高階関数**の中身がブラックボックスになっているので、パーサコンビネータに対する最適化は困難である。
- 一方、パーサコンビネータを**DSL**として定義すると、最適化や部分評価が適用でき、効率的な解析が可能になる。



研究目標

- 静的に最適化できる文脈依存パーサコンビネータを提案
- 正規言語を解析するパーサコンビネータを最適化

文脈依存性

モナド式

解析結果
 $p_0 \gg= \lambda x. p_1$
次のパーサ

- p_1 に対する最適化は困難

($x :: \text{Int}$)は `number` の解析結果

```
number >>= \x ->
  if isPrime x
  then read "is prime"
  else read "isn't prime"
```

実行前の最適化は困難

本研究の定式化

$\mathcal{P} ::= \dots$

- x
- $\lambda x. \mathcal{P}$
- $\mathcal{P}_1 \mathcal{P}_2$
- if** $\mathcal{P}_1$ **then** $\mathcal{P}_2$ **else** $\mathcal{P}_3$

App

```
(Lam (If (Var x `Map` isPrime)
  (read "is prime")
  (read "isn't prime")))
```

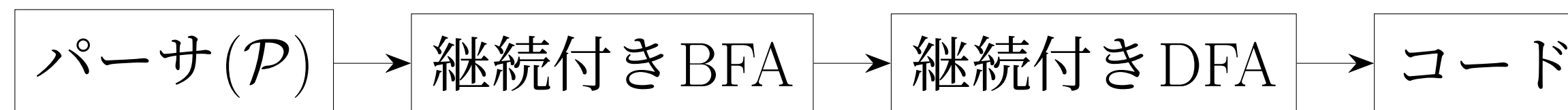
- 実行前の最適化は可能
- モナド方式と同じ程度 of 文脈依存性を持つ

正規言語に対する最適化

- 目的** 正規言語を解析するパーサコンビネータをDFAに変換し、コードを生成する。
ナイーブなパーサコンビネータ: 時間 $O(2^n)$ 空間 $O(n)$
→ 最適化したパーサコンビネータ: 時間 $O(n)$ 空間 $O(1)$
- 対象** $\mathcal{P} ::= \perp \mid \uparrow a \mid \#c \mid \mathcal{P}_1 \cdot \mathcal{P}_2 \mid \mathcal{P}_1 / \mathcal{P}_2 \mid x \mid \mu x. \mathcal{P} \mid \mathcal{P}[f]$ [Krishnaswami 2019]
 $\emptyset \quad \varepsilon \quad \{c\} \quad \text{接続} \quad \text{選択} \quad \text{変数} \quad \text{不動点} \quad \text{結果変換}$
- 条件** 左再帰でないかつ末尾再帰となるもの
例 $\mu x. (\#a \cdot x)[\text{cons}] / \uparrow \text{nil}$ 反例 $\mu x. x, \mu x. \#(\cdot x \cdot \#)$

問題点

- 優先度付き選択: $L(a|ab) = \{a, ab\}$ $L(a/ab) = \{a\}$
解決策 Boolean Finite Automaton(BFA)に変換 [Chida & Kuramitsu 2017]
- 解析結果の処理: リターン($\uparrow a$)とマップ($\mathcal{P}[f]$)
解決策 BFAなどを拡張して、**継続付きステートマシン**を提案



継続付きステートマシン

継続付きDFA $M = (Q, \Sigma, \delta, q_0, F, k_0, \text{return})$



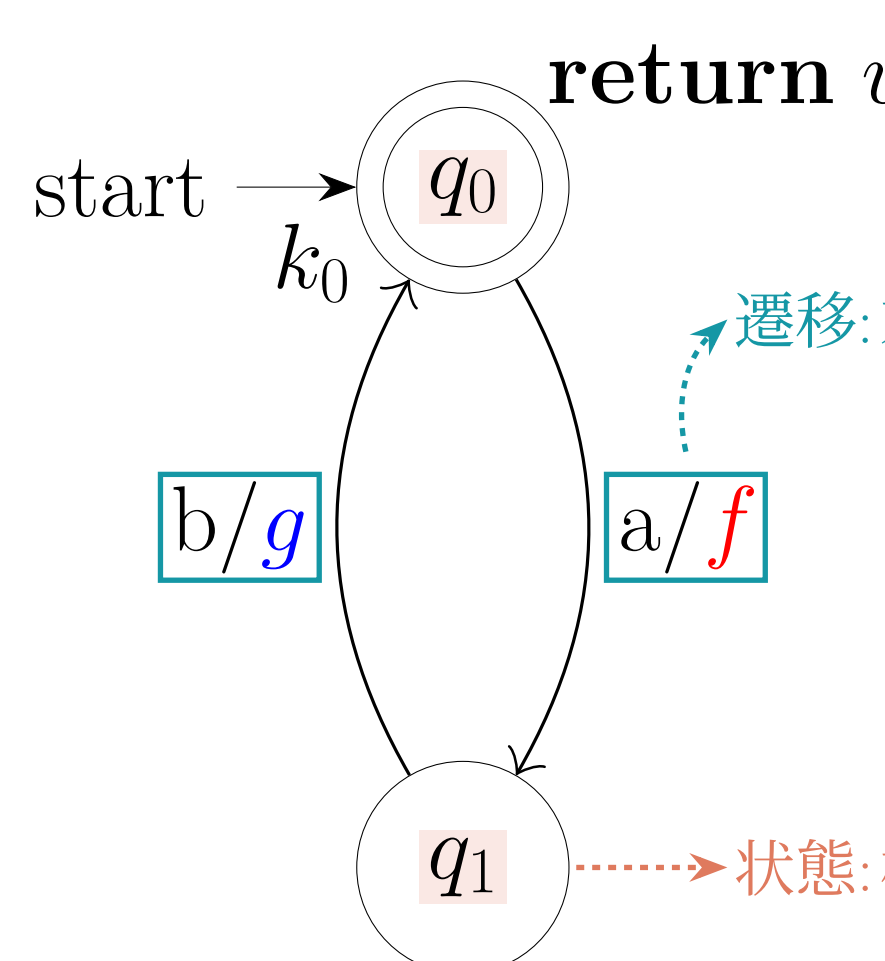
具体例

abが交互に並ぶ列(i.e. $(ab)^*$)
 $\mu x. (\#a \cdot (\#b \cdot x)[g])[f] / \uparrow v$

DFAwK(右図)

入力“abab”に対して、
 $q_0 \xrightarrow{a} (q_1, f) \xrightarrow{b} (q_0, g)$
 $\xrightarrow{a} (q_1, f) \xrightarrow{b} (q_0, g)$ ✓

解析結果 = $k_0(f(g(f(gv))))$



```
let
  q0 s k =
    case s of
      'a' : s_ -> q1 s_ (k o f)
      -       -> Just (k v)
  q1 s k =
    case s of
      'b' : s_ -> q0 s_ (k o g)
      -       -> Nothing
in \s -> q0 s k0
```

解析結果 = $(k_0 \circ f \circ g \circ f \circ g) v$

予備実験

今後の課題

比較対象

ナイーブに部分評価したパーサコンビネータ [Jonnalagedda et al. 2014]

実験1(左図)

01列に1110が出現する回数を数える

- 両方も線形時間で解析する
- 平均的な速度向上は2.11倍である

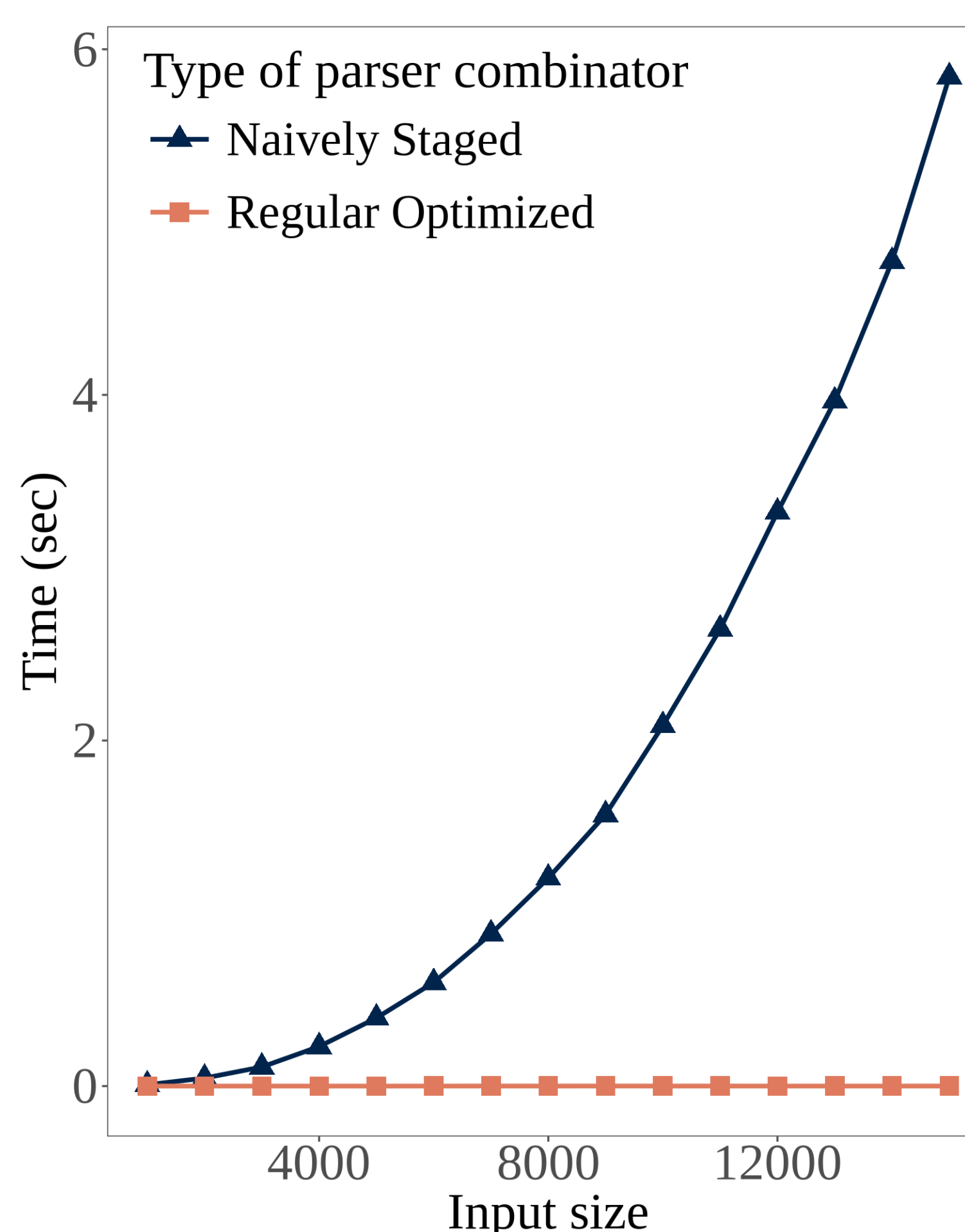
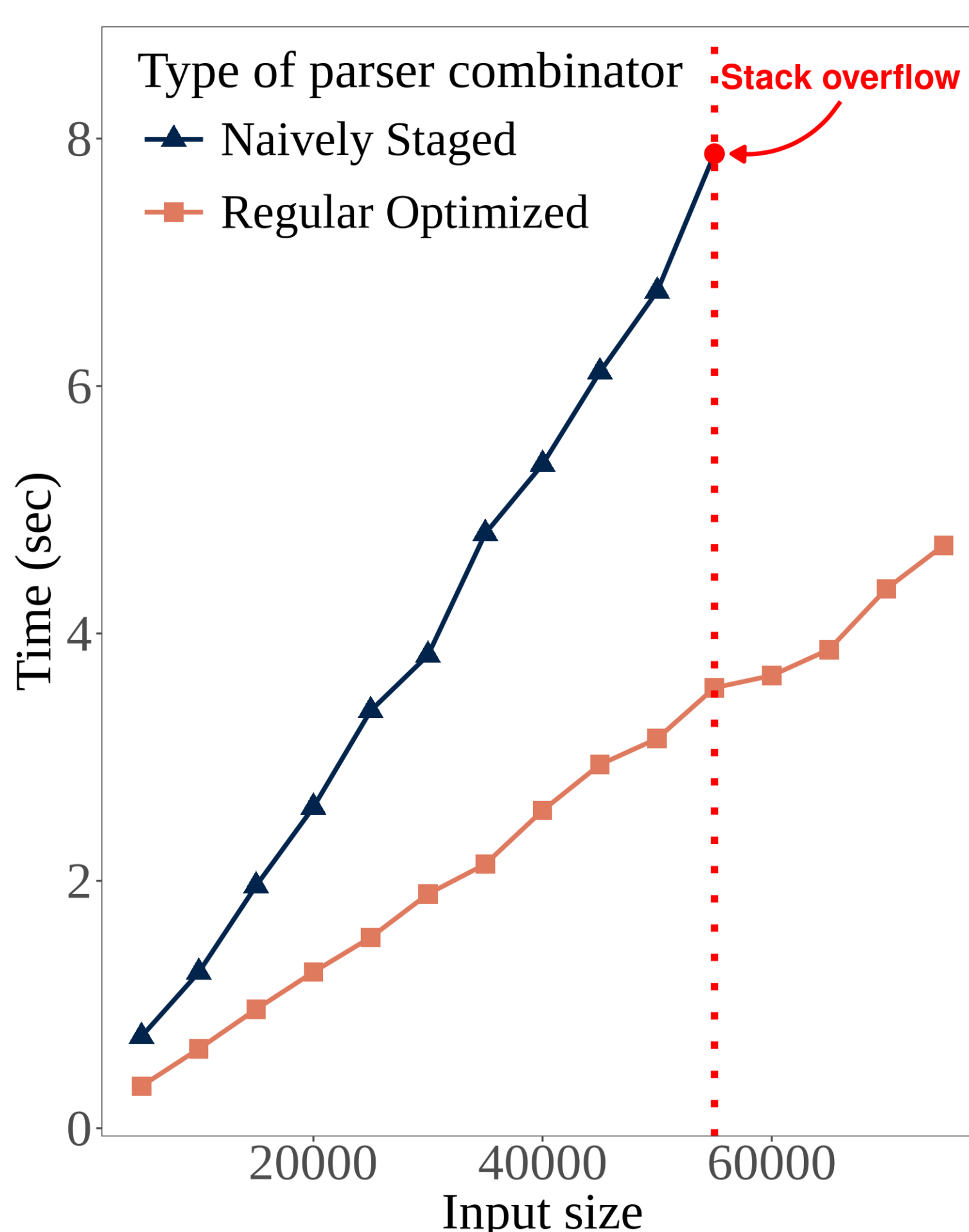
	BFAwK	DFAwK	最小化
状態数	72	17	7
遷移数	38	34	14

実験2(右図)

$S \rightarrow A \mid xS \mid z$
 $A \rightarrow xA \mid y$

- ナイーブ: $O(n^2)$; 本研究: $O(n)$.

	BFAwK	DFAwK	最小化
状態数	32	6	3
遷移数	24	9	3



ハイブリッド最適化

Regex(本研究)とLL(1)[Krishnaswami 2019]の組み合わせ。

最適化できない $\mathcal{P}_1 / \mathcal{P}_2$

Regex最適化可能 $\mathcal{P}_1$ LL(1)最適化可能 $\mathcal{P}_2$

部分評価

```
case
  P1
of Just res -> Just res
   Nothing -> P2
```

Regex最適化して生成したコード

LL(1)最適化して生成したコード

- 共に適用可能な最適化を検討
- 効率とコードサイズのトレードオフを評価